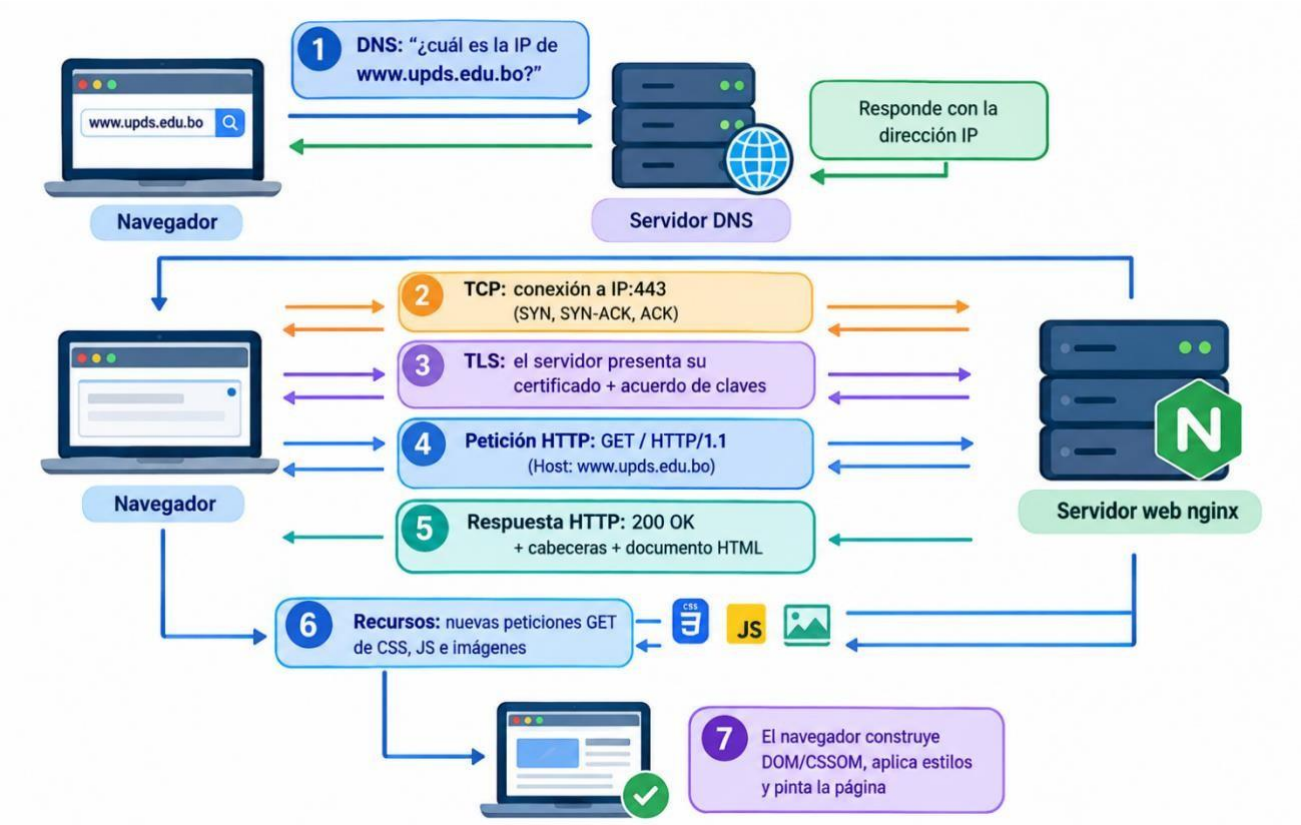
Laboratorio 1 — Hoja de trabajo: el viaje de una petición HTTP

Nombre: Matias Ezequiel Flores Mendez Fecha: 09/10/2026

Grupo: Los Rezagados

Parte A — Diagrama cliente-servidor

Dibuja (en papel, Excalidraw o con texto) el recorrido de una visita a `https://www.upds.edu.bo`: navegador → DNS → servidor → respuesta. Marca dónde intervienen: IP, puerto, TLS, petición, respuesta, HTML, recursos.



Dónde intervienen:

IP: la entrega el servidor DNS en el paso 1; es la dirección a la que se conecta el navegador.

Puerto 443: puerto estándar de HTTPS; se usa al abrir la conexión TCP (paso 2).

TLS y certificado: en el paso 3 el servidor se identifica con su certificado digital y ambos acuerdan las claves con las que se cifrará todo lo siguiente.

Petición y respuesta: pasos 4 y 5; la respuesta trae el código de estado, las cabeceras y el HTML.

HTML y recursos: el HTML llega en el paso 5; al leerlo, el navegador descubre y pide CSS, JS e imágenes (paso 6).

Parte B — Explora el servidor de práctica

Inicia el servidor: `python servidor_eco.py` y deja esa terminal abierta. En OTRA terminal:

#	Comando	¿Qué código de estado devolvió?	¿Qué cabecera de respuesta es la más importante aquí y por qué?
1	`curl -v http://127.0.0.1:8080/`	200 OK	**Content-Type: text/html; charset=utf-8.** Avisa al cliente que el cuerpo es HTML en UTF-8 y no texto plano ni un binario; sin eso no sabría cómo mostrarlo. Con -v se ven la petición (>) y la respuesta (<).
2	`curl -I http://127.0.0.1:8080/antiguo`	301 Moved Permanently	**Location: /.** Dice a qué dirección se movió el recurso de forma permanente. Con -I curl envía HEAD (solo cabeceras) y se detiene en el 301.
3	`curl -L -v http://127.0.0.1:8080/antiguo`	301 → 200 OK	**Location: /.** Con -L curl lee esa cabecera y hace por sí mismo una segunda petición GET a /, que sí responde 200. En el -v se ven las dos peticiones y se reutiliza la misma conexión.
4	`curl -i http://127.0.0.1:8080/privado`	403 Forbidden	**No hay una cabecera clave: lo importante es la línea de estado 403.** El servidor entendió la petición pero se niega a atenderla; no pide credenciales, y por eso no es un 401.
5	`curl -i http://127.0.0.1:8080/no-existe`	404 Not Found	**Content-Type: text/html** (con Content-Length: 64). La ruta no existe, pero el servidor responde con una página HTML de error propia del colegio en vez de dejar al usuario sin explicación.
6	`curl -i -X POST -H "Content-Type: application/json" -d "{\"curso\":\"6to\"}" http://127.0.0.1:8080/eco`	201 Created	**Content-Type: application/json; charset=utf-8.** Indica que el cuerpo no es una página sino un JSON que repite lo que envié (método, cabeceras y el cuerpo {"curso":"6to"}).
7	`curl -o NUL -w "%{http_code} %{time_total}s\n" http://127.0.0.1:8080/lento`	200 OK (≈ 2,00 s)	**No hay cabecera de respuesta clave; lo que se mira es la métrica %{time_total} de curl,** que no es una cabecera. Midió los 2 s de espera que el servidor provoca con time.sleep(2).

(En Linux/macOS usa `-o /dev/null` en lugar de `-o NUL`.)

Comparación de /antiguo con -I y con -L: con -I curl hace una sola petición (HEAD) y muestra el 301 sin seguirlo. Con -L hace dos: primero /antiguo (301), toma la cabecera Location y enseguida pide / (200). En la terminal del servidor se ven dos líneas de registro, una por cada petición.

Parte C — Cookies

1. Ejecuta dos veces: curl -i -c galletas.txt -b galletas.txt http://127.0.0.1:8080/cookie

   Resultado: ejecuté el comando dos veces; en la primera aparece 1 visita y en la segunda sube el contador.

2. ¿Qué número mostró la segunda vez? ¿Dónde guardó curl la cookie?

   Respuesta: la segunda vez mostró "Visitas registradas con tu cookie: 2". curl guardó la cookie en el archivo de texto galletas.txt, con esta línea:
   `#HttpOnly_127.0.0.1  FALSE  /  FALSE  1791308633  visitas  2`

   Ahí quedan el dominio (127.0.0.1), la ruta (/), el momento en que vence (en formato Unix) y el par visitas = 2. En la segunda petición curl lee ese archivo (-b) y envía la cookie de vuelta.

3. ¿Qué significan `HttpOnly`, `SameSite=Lax` y `Max-Age=3600`?

   HttpOnly: impide que JavaScript lea la cookie (no se puede ver con document.cookie). Así, si alguien logra inyectar un script malicioso (ataque XSS), no puede robar la cookie.

   SameSite=Lax: el navegador no envía la cookie en peticiones que se originan en otro sitio (por ejemplo, un formulario POST puesto en una web ajena), pero sí cuando el usuario llega haciendo clic en un enlace. Ayuda contra los ataques CSRF.

   Max-Age=3600: tiempo de vida de la cookie en segundos; 3600 s son 1 hora, y pasado ese tiempo el cliente la borra.

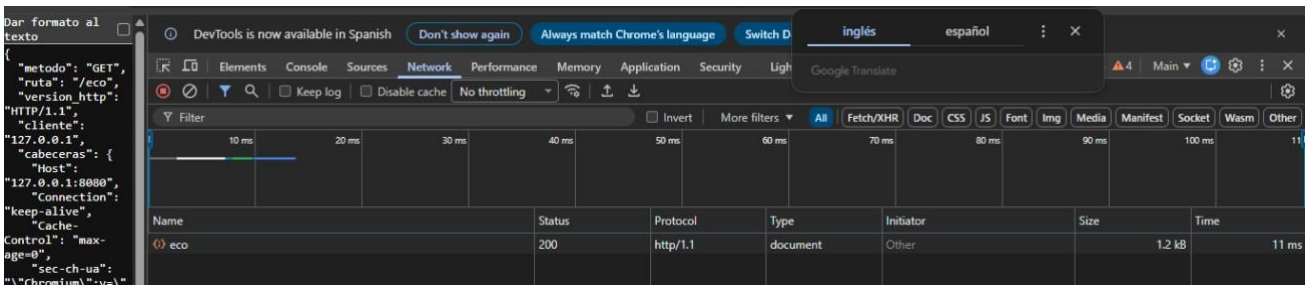
Parte D — Lo mismo en el navegador

Abre `http://127.0.0.1:8080/` con DevTools > Red (Network). Visita /cookie, /antiguo y /lento. Anota: el código de cada petición, la columna Time de /lento y dónde ves la cookie (Application > Cookies).

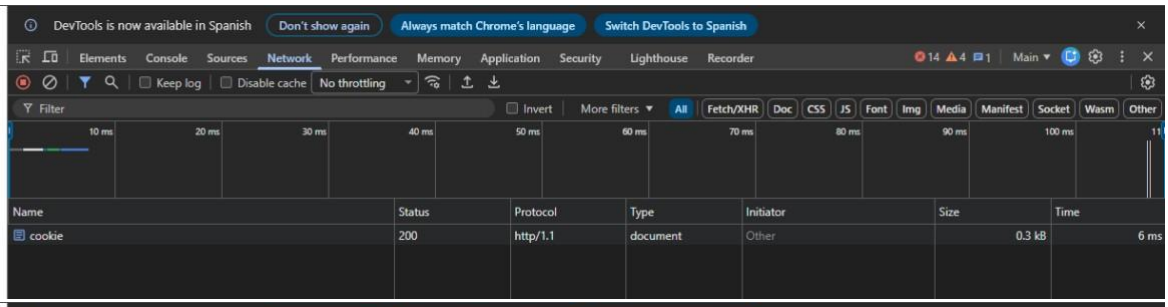
Códigos de las peticiones: / → 200 (protocolo http/1.1). /cookie → 200, con la cabecera SetCookie. /antiguo → aparecen dos filas: primero 301 y enseguida / con 200, porque el navegador sigue la cabecera Location por sí solo. /lento → 200.

Columna Time de /lento: ≈ 2,0 s. En la pestaña Timing casi todo el tiempo es "Waiting for server response", es decir, el navegador solo esperaba al servidor (el retraso es del servidor, no de la red).

Dónde se ve la cookie: Application > Storage > Cookies > http://127.0.0.1:8080. Ahí aparece visitas con su valor, Path / y la casilla HttpOnly marcada (✓).



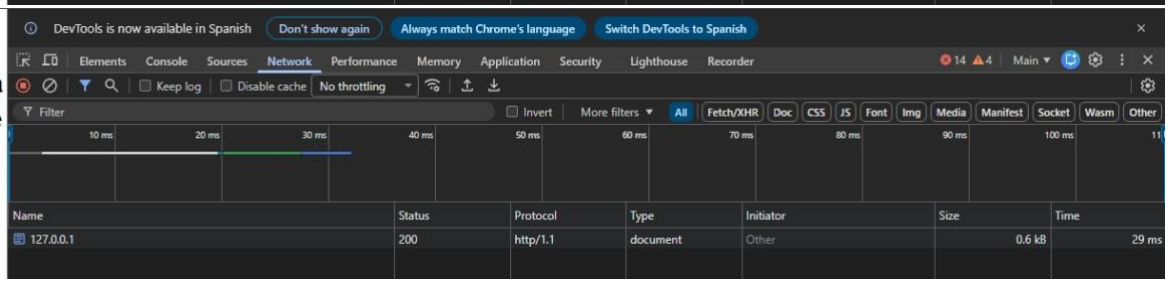
Visitas registradas con tu cookie: 12



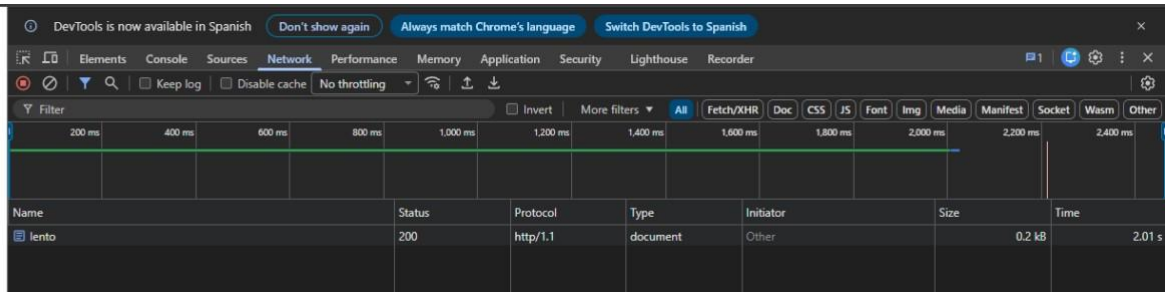
Unidad Educativa Horizonte

Servidor de práctica de Tecnología Web I. Prueba estas rutas:

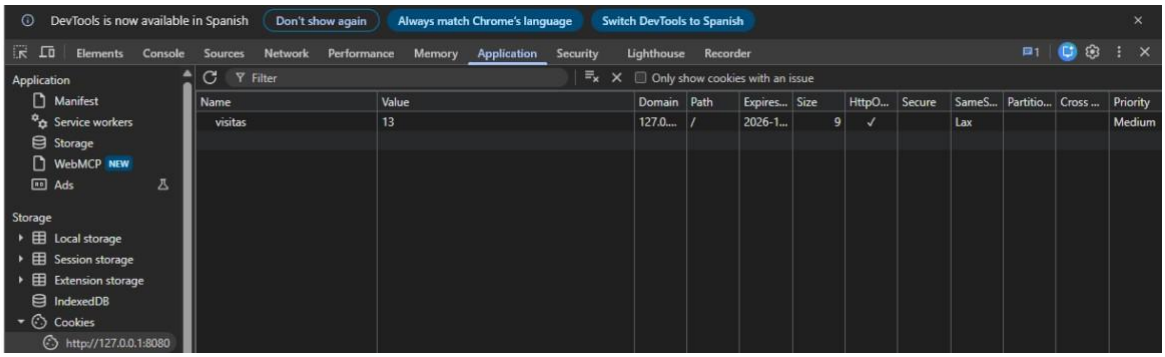
- /eco
- /cookie
- /antiguo



Respuesta después de 2 segundos.



Visitas registradas con tu cookie: 13



Parte E — Un sitio real

`curl -sI https://www.google.com` y `curl -sI https://www.upds.edu.bo` (si hay conexión). ¿Qué servidor web dicen usar (cabecera Server)? ¿Usan HTTP/2? ¿Qué cabeceras de seguridad ves?

Sitio	Cabecera Server	¿HTTP/2 o superior?	Cabeceras de seguridad
https://www.upds.edu.bo	nginx (con PHP: lo delatan XHttpd-Modphp y el Link a wpjson de WordPress)	curl en Windows muestra HTTP/1.1; el navegador, en la columna Protocol, muestra h2	No envía Strict-TransportSecurity ni Content-SecurityPolicy
https://www.google.com	gws (Google Web Server)	Anuncia HTTP/3 con Alt-Svc: h3=":443"; el navegador usa h2 o h3	Content-Security-Policy-ReportOnly, X-Frame-Options: SAMEORIGIN y cookies con Secure, HttpOnly y SameSite
https://es.wikipedia.org	HTTP/1.1 301 Moved Permanentl	HTTP/1.1	Envía Strict-Transport-Security

http://127.0.0.1:8080 (servidor de práctica)	HorizonteEco/1.0 Python/3.14.7	HTTP/1.1	Ninguna: es un servidor local de laboratorio

Lo que cambia según la herramienta: el curl de Windows (Schannel) no trae soporte HTTP/2 y siempre muestra 1.1, aunque el sitio sí lo use; el navegador, con la columna Protocol, muestra el protocolo realmente negociado. Por eso conviene comprobar con ambas herramientas.

Reflexión (3-4 líneas)

¿Qué aprendiste que no sabías sobre lo que pasa cuando abres una página?

Lo que más me llamó la atención es que la herramienta cambia lo que uno ve: con curl hacia /antiguo recibo solo un 301 con un mensaje de mudanza, mientras que el navegador sigue la redirección sin avisar y hace dos viajes de red en lugar de uno. También me sorprendió comprobar que un sitio institucional como el de la UPDS no envía las cabeceras HSTS ni CSP, y que la misma página puede aparecer como HTTP/1.1 o h2 según con qué herramienta la mire.